BITTRADER

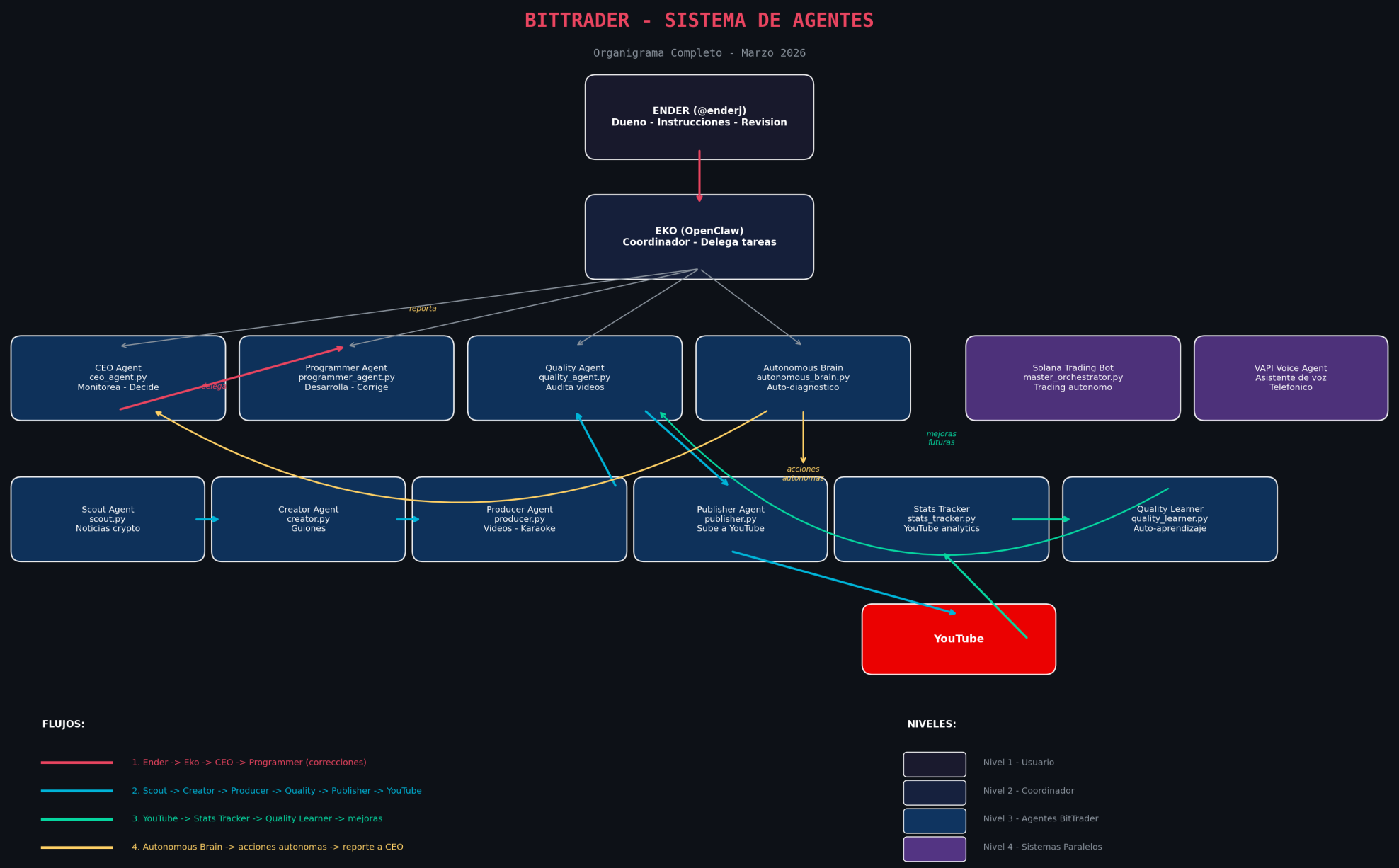
Sistema de Agentes - Organigrama Completo

Generado: 2026-03-16 09:28

Autor: Programmer Agent | Solicitado por: Ender (@enderj)

Este documento presenta la arquitectura completa del sistema de agentes BitTrader, incluyendo el organigrama visual, tabla de funciones por agente, analisis del estado actual del sistema y recomendaciones de mejora.

1. ORGANIGRAMA DEL SISTEMA



2. TABLA DE FUNCIONES POR AGENTE

Nivel	Agente	Archivo	Responsabilidades
1	Ender (@enderj)	-	Dueno del sistema. Da instrucciones, revisa resultados, aprueba publicaciones.
2	Eko (OpenClaw)	-	Coordinador general. Recibe instrucciones de Ender y las delega al equipo de agentes.
3	CEO Agent	ceo_agent.py	Monitorea el sistema, toma decisiones estrategicas, detecta issues y delega al Programmer.
3	Programmer Agent	programmer_agent.py	Desarrolla codigo, corrige bugs, implementa mejoras tecnicas solicitadas por CEO o Eko.
3	Quality Agent	quality_agent.py	Audita videos antes de publicar. Valida calidad visual, audio, subtitulos y branding.
3	Autonomous Brain	autonomous_brain.py	Auto-diagnostico del sistema. Ejecuta acciones autonomas sin intervencion humana.
3	Scout Agent	scout.py	Monitorea noticias crypto en tiempo real. Genera ideas de contenido relevante.
3	Creator Agent	creator.py	Escribe guiones para videos basados en ideas del Scout y tendencias del mercado.
3	Producer Agent	producer.py	Genera videos con clips, subtitulos karaoke, branding. Incluye ken_burns_producer.py.
3	Publisher Agent	publisher.py	Sube videos a YouTube, gestiona metadatos, thumbnails y scheduling.
3	Stats Tracker	stats_tracker.py	Seguimiento de estadisticas YouTube a 24h/48h/7d. Cierra el loop de aprendizaje.
3	Quality Learner	quality_learner.py	Sistema de auto-aprendizaje. Registra lecciones de errores para mejorar produccion.
4	Solana Trading Bot	master_orchestrator.py	Bot de trading autonomo en Solana. Operaciones cripto en tiempo real.
4	VAPI Voice Agent	- (externo)	Asistente de voz telefonico. Interfaz de voz para consultas y operaciones.

3. FLUJOS PRINCIPALES

Flujo 1: Correccion de Bugs

Ender reporta un problema a Eko. Eko coordina con el CEO Agent, quien analiza el issue y lo delega al Programmer Agent para correccion. El Programmer implementa el fix y el CEO valida el resultado.

Ende

Flujo 2: Pipeline de Contenido

El Scout detecta una noticia crypto relevante. El Creator genera un guion. El Producer crea el video con clips, subtitulos y branding. El Quality Agent audita el video antes de publicar. El Publisher lo sube a YouTube.

Scou

Flujo 3: Feedback Loop de Aprendizaje

Una vez publicado, el Stats Tracker monitorea el rendimiento a 24h, 48h y 7 dias. Los datos alimentan al Quality Learner, que extrae lecciones y las aplica para mejorar la produccion futura automaticamente.

YouT

Flujo 4: Autonomia del Sistema

El Autonomous Brain ejecuta diagnosticos periodicos sin intervencion humana. Detecta problemas, ejecuta acciones correctivas y reporta al CEO Agent. Esto permite que el sistema se auto-repare en gran medida.

Auto

4. ANALISIS DEL PROGRAMMER AGENT

Lo que esta BIEN:

- Pipeline completo de contenido: desde noticia hasta YouTube publicado, todo automatizado.
- Feedback loop cerrado: Stats Tracker + Quality Learner permiten mejora continua sin intervencion.
- Quality gate antes de publicar: el Quality Agent previene errores visibles al publico.
- Autonomous Brain da resiliencia: el sistema se auto-diagnostica y corrige sin humanos.
- Arquitectura modular: cada agente tiene una responsabilidad clara y un archivo dedicado.
- Sistemas paralelos (Solana Bot, VAPI) operan independientemente sin afectar el pipeline principal.

Lo que FALTA:

- No hay sistema de retry/recovery centralizado si un agente falla a mitad del pipeline.
- No existe dashboard en tiempo real para ver el estado de todos los agentes simultaneamente.
- Falta un sistema de alertas unificado (actualmente cada agente maneja sus propios logs).
- No hay tests automatizados para validar el comportamiento de cada agente individualmente.
- Falta documentacion formal de APIs internas entre agentes (contratos de datos).

Que se puede MEJORAR:

- El Producer podria beneficiarse de A/B testing automatico de thumbnails.
- El Quality Agent podria usar un scoring system numerico en vez de pass/fail.
- La comunicacion entre agentes es via archivos JSON - un message bus seria mas robusto.
- El Stats Tracker podria correlacionar metricas con variables del guion para optimizar contenido.
- El Autonomous Brain podria tener niveles de severidad para escalar issues criticos a Ender.

5. TOP 5 RECOMENDACIONES DE MEJORA

1. Agent Health Dashboard

Crear un dashboard web (FastAPI + HTML) que muestre el estado en tiempo real de cada agente: ultimo heartbeat, tareas pendientes, errores recientes, y metricas de rendimiento. Esto elimina la necesidad de revisar logs individuales.

IMPA

2. Message Bus Centralizado

Reemplazar la comunicacion por archivos JSON con un sistema de mensajeria (Redis Pub/Sub o similar). Esto reduce race conditions, mejora la trazabilidad y permite event-driven architecture.

IMPA

3. Pipeline Error Recovery

Implementar un sistema de retry con backoff exponencial y dead-letter queue para cuando un agente falla. Si el Producer falla, el video no se pierde - entra en cola de recuperacion.

IMPA

4. A/B Testing de Thumbnails

El Producer genera 2-3 variantes de thumbnail. El Publisher sube la mejor segun el Quality score, y el Stats Tracker compara rendimiento para retroalimentar al Quality Learner.

IMPA

5. Agent Unit Tests

Crear tests automatizados para cada agente con datos mock. Esto permite validar que cambios en un agente no rompen el pipeline. Integrar con CI/CD para validacion continua.

IMPA